Verifiable RAG over SEC Filings: Claim Provenance, Faithfulness, and Error Attribution

A reading companion to the sec_llm_project claim-provenance system: method, implementation, data, results, and the literature behind the design. Prepared July 2026.

1. The problem this system solves

Retrieval-Augmented Generation (RAG) couples a retriever over a document corpus with a generative language model: the model answers *conditioned on retrieved passages* rather than only from knowledge stored in its weights during training (its "parametric memory," in the literature's term [1, 2]). SEC filings are a natural fit for this design: they are long, numerous, and authoritative, and the answer to "what liquidity risks does Tesla disclose?" should come from Tesla's actual 10-K, not from whatever the model absorbed during pretraining.

The design hides a measurement problem. A fluent, well-cited RAG answer can still be wrong in ways invisible to the reader: it can cite the wrong fiscal year, present a risk *disclosure* as an occurred *event*, drop the hedging language that made a forward-looking statement legally careful, attach a real citation to a claim the cited text does not support, or answer confidently from tangential passages when the truly relevant disclosure was never retrieved. Aggregate quality scores do not distinguish these failures. The reason is structural, not accidental: a single score averages over all failure modes, so two systems (or two questions) with the same score can fail in entirely different ways. The fixes live in different subsystems — a retrieval failure calls for better indexing or query handling, a generation failure for better prompting or gating, a judging failure for a better verifier — so a metric that cannot attribute an error to a subsystem cannot guide engineering effort. This is the practical argument for *diagnostic* evaluation over *scoreboard* evaluation, and it is why this project treats the explanation artifact, not the score, as the primary output.

The response is a **claim-provenance DAG**: for every answer, an explicit directed acyclic graph from *evidence spans* (retrieved filing chunks with full provenance metadata) through *atomic claims* (judged statements extracted from the answer) to a *final or rejected conclusion*. The graph is built only from what the pipeline actually did — no post-hoc rationalization — and it is the unit of explanation: aggregate metrics say **what** the system scores; the DAG says **why** a particular answer earned or lost trust.

This document proceeds paper-style: the system (§2) and its implementation with rationale (§3), the data (§4), results on the full gold-set run and negative controls (§5), discussion (§6), and then the research literature each design decision draws on (§7–§15), a synthesis (§16), and a suggested reading order (§17).

2. The system under study, in one page

The whole system runs on a single machine; no data is sent to any outside service. Before it can answer anything, it has to break the filings into pieces small enough to work with. A full annual report can run to hundreds of pages — far more text than a language model can read at once — so each filing is cut into short passages of a few hundred words. Following standard usage, we call each passage a **chunk**. The cut points are chosen deliberately rather than every N characters blindly: SEC filings are divided into numbered sections — "Item 1A. Risk Factors", "Item 7. Management's Discussion and Analysis", and so on — and the system splits on those section headers, so a chunk rarely mixes two unrelated topics. Each chunk keeps a label recording where it came from: the company's stock ticker, the form type (an annual 10-K, a quarterly 10-Q, or an event-driven 8-K), the filing date, the fiscal year, and the section it was cut from. That labeling is what later lets every sentence of an answer be traced to a specific place in a specific filing.

When a question arrives, the first job is to find, among the roughly 280,000 chunks, the handful most likely to contain the answer. This step is called **retrieval**. The system judges relevance two complementary ways, because each catches what the other misses. The first is **keyword matching**: score a chunk by how many of the question's words appear in it, weighting rarer words more heavily — the long-standing algorithm for this is BM25 [3]. Keyword matching is excellent on exact terms (tickers, acronyms, defined phrases) but blind to synonyms. The second is **meaning matching**: a separate model (Nomic [4]) turns each chunk into a list of 768 numbers — an **embedding** (also called a **vector**) that encodes the passage's meaning so that passages about similar things get similar number-lists — and the system finds the chunks whose vectors sit closest to the question's vector. This catches paraphrase ("liquidity concerns" vs. "ability to meet obligations") that keyword matching would miss. Those vectors are held in a database built for fast closest-vector lookup (Chroma [5], which uses an index called HNSW so it need not compare against all 280,000 vectors one at a time). Each method returns its own ranked list of chunks; the two lists are merged into one by a simple rank-combining rule — reciprocal-rank fusion [6] — that rewards chunks either method ranked highly.

Two numbers control how wide this net is cast, and they answer different questions. **Candidate pool = 60**: each method first pulls its own top 60 chunks independently — deliberately more than needed, cheap to compute, and wide enough that a chunk ranked 40th on keywords but 3rd on meaning still gets a chance to be merged in. This is an engineering default. **Final selection = 8**: after merging, only the 8 best chunks are passed on to write the answer. Eight is the number the two methods' merge weights (60% keyword, 40% meaning) were actually tuned for, by trying weight combinations and measuring how often the known-correct passage landed in the top 8 across a 260-question test set (the best result came at exactly 60/40). So 60 is "cast a wide net cheaply" and 8 is "the setting we actually optimized."

Those 8 chunks are handed to a language model — the **generator** (Gemma 3 12B, run locally through a tool called Ollama) — which writes the answer. It runs deterministically (the same question always yields the same answer), is capped at roughly a page of output, and is required to attach a bracketed citation like `[3]` to each statement, pointing at the specific chunk that statement came from. (Throughout, sizes are measured in **tokens** — the sub-word units models read and write; a token is about four characters of English, so a page of text is on the order of a thousand tokens.) The system can

abstain — decline to answer — at three points: before it even generates, when the retrieved chunks look too weak to support any answer; after generation, when the verification step (described below) finds too few of the answer's claims actually grounded; and when the generator itself reports that it lacks the evidence to answer. Each of these is made precise in the paragraphs that follow. Every retrieval and answer event is appended to a JSONL audit log (a plain text file, one JSON record per line, that accumulates a permanent record of every question and decision the system makes — the raw material an auditor or a debugging session reads).

Verification is the distinctive layer, and the reason the answer can be trusted at all. Once the generator has written an answer, a second language model — the **judge** — checks it. The judge is deliberately a *different* model from a different maker (Mistral 7B, where the generator is Gemma; the system refuses at startup to let a model grade its own output, for the same reason you would not let a student mark their own exam). The judge does two things. First it breaks the answer into its separate factual statements — its **atomic claims**, each asserting a single checkable thing. Then it labels each claim against the retrieved chunks as **SUPPORTED** (the chunks back it up), **PARTIAL** (partly backed), **UNSUPPORTED** (not found in the chunks), or **CONTRADICTED** (the chunks say the opposite); for every claim it calls supported, it must name the chunk that supports it and copy out the exact sentence.

Running the judge is slow enough that the system waits up to 600 seconds for each verdict before treating the call as failed — a deadline that looks absurdly generous until you see where the time goes. The total time the system waits has two parts, and only the second is "thinking": first the judge model must be *loaded into memory*, then it *generates*. On a 16 GB machine the generator (Gemma 12B, ~7 GB) and the judge (Mistral 7B, ~4 GB) cannot both stay resident at once, so answering a question and then verifying it means loading the generator, unloading it, and loading the judge from disk — and that load alone can take tens of seconds before the judge produces its first token. The 600 seconds therefore has to cover *load time plus* the generation of a long, quote-bearing verdict; a deadline sized only for the generation part would abort during a perfectly normal model swap and discard a valid verification as if it had crashed.

The system does not take the judge's verdicts on trust. After the judge returns, a set of purely mechanical checks re-examines each verdict — plain string and number comparisons, no second model, so they cannot themselves hallucinate. The central one: a claim the judge marked SUPPORTED is allowed to stand as supported only if the quote the judge attached to it is found *word-for-word* in the chunk it cited; if that quote is not actually in the chunk, the claim is automatically downgraded to unsupported. In other words the judge is trusted to *propose* that a claim is grounded, but a claim is only *accepted* as grounded when its evidence checks out literally. These checks are spelled out in §3.4.

Once every claim has a checked verdict, the system decides whether the answer as a whole is grounded enough to deliver. It measures this by the fraction of the answer's claims that survived as supported — call this the answer's *groundedness*. If that fraction falls below a set threshold (here 0.6: fewer than 60% of the claims supported), the answer is treated as too weakly grounded to stand as written. Rather than discard it outright, the system makes one corrective attempt: it shows the generator exactly which claims failed and asks it to rewrite the answer keeping only the supported ones, then re-runs the whole

verification on that rewrite. If the rewrite still falls below the threshold, the system declines to answer at all — it *abstains* — and records why, on the principle that no answer is better than a poorly grounded one. The DAG is assembled from these artifacts and exported as JSON, Graphviz (PDF/PNG for slides), and interactive HTML; downstream tools tabulate per-claim mistakes, attribute each question's dominant error source, and aggregate RAGAS-style corpus metrics computed with this local machinery.

Both the claim *decomposition* and the claim *identification* are done by the judge model itself, not by the generator or by a separate parser — a point worth making precisely because it is easy to assume otherwise. The generator writes an answer with `[n]` citations and nothing more; it never labels which of its sentences are "the claims." A second prompt then hands the judge the finished answer text and asks it to split that text into standalone factual statements and grade each one against the retrieved evidence (see the prompt in §3.4). This means the claims are the *judge's own segmentation* of the answer — there is no access to the generator's internal reasoning, and nothing in this design claims there is. What can be checked mechanically is only the second half: once the judge names a claim's supporting evidence and quote, that quote is verified as an actual substring of the cited chunk (§3.4). The decomposition step itself — did the judge split the answer into claims sensibly, without merging two statements or inventing one that was not really there — is not independently verified, and is the one place in this pipeline where a wrong judgment could pass through undetected.

3. The implementation, and why it is built that way

This section walks the code path of one question, pausing where a design decision encodes a principle. File references are to the repository layout (`src/sec_llm_project/sec_intel/…`).

3.1 Configuration: real backends by default, mock by explicit request

Everything is wired from one `AppConfig` dataclass tree (`core/config.py`); providers are chosen by string, never by code edits. The consequential decision is what the *defaults* are:

```
@dataclass
class LLMConfig:
    # provider: ollama (real, local — the default) | mock (deterministic offline
    # placeholder). The default is a REAL backend on purpose: a mock answer is not
    # a real answer, so mock is never a silent fallback — select it explicitly via
    # AppConfig.offline(), config/offline.yaml, provider=mock, or `sec-intel-mock`.
    provider: str = "ollama"
    model: str = "gemma3:12b"
```

The default provider is the real model, not the mock, on purpose: a mock fallback would let a missing `--config` produce a fluent, cited, *fake* answer — the one output an evaluation system must never manufacture. So the offline stack (mock model + hashing embeddings + in-memory index, for tests and CI) is reachable only through an explicit `AppConfig.offline()` or the separate `sec-intel-mock` binary, and the real tool refuses to run without a config rather than quietly degrading. The rule generalizes: **placeholder behavior must be unreachable by accident**.

3.2 Chunking and identity: reproducibility as a primary key

Chunks split on the `Item N.` section boundaries described in §2 and carry the full provenance tuple (ticker, form, date, item number, and so on). There are two chunk sizes in this project, for two different purposes, not two independent systems: the repository's default configuration (1,800 characters, 200-character overlap between consecutive chunks so a sentence split across a boundary still appears whole in at least one chunk) is meant for small experiments, tests, and quick indexing runs. The actual 281,777-chunk production index used for the results in §5 was built at 3,600 characters per chunk — double the default — because embedding is the slow step in indexing (§3.3), and doubling the chunk size halves the number of chunks that must be embedded, at the cost of each chunk covering more text (and therefore being a slightly blunter unit of retrieval). Both sizes use the same chunker and the same `Item N.` splitting logic; only the target character count differs. Chunk identity is content-derived:

```
basis = "|".join(
    [metadata.ticker, filing_key, metadata.fiscal_year, metadata.filing_type,
     item_number, str(ordinal), text_head])
digest = hashlib.sha1(basis.encode("utf-8")).hexdigest()[:12]
return f"{metadata.ticker}:{item_number}:{ordinal}:{digest}"
```

Why a deterministic id rather than a random one (e.g. a UUID generated fresh each time): the same corpus and config must produce the *exact same id* for the *exact same chunk* on every machine and every re-run — the id is computed from the chunk's own content and metadata (via the SHA-1 hash above), not assigned arbitrarily. This one property pays for itself in three concrete ways.

Idempotent indexing. "Upsert" is database shorthand for "update if it already exists, insert if it doesn't" — the operation is safe to repeat. Because a chunk's id never changes between runs, re-indexing the same filing twice does not create a duplicate entry; it just overwrites the same row with the same content. Concretely, this means the 16-hour overnight indexing run described in §3.6 can be killed and restarted at any point: on restart, every chunk the process already indexed upserts to the identical id it had before, so no chunk is embedded twice and no chunk is lost.

Cross-artifact joins. The same chunk shows up in several different files over the life of a question — the audit log, a citation on an answer, a node in a DAG, and a row in the Chroma vector store. Because the id is computed the same way everywhere, the string `TSLA:1A:27:4c0278cef4c1` seen in any one of those places refers to the exact same chunk in all the others; there is no separate lookup or mapping table to keep in sync.

Diffable evaluation. "Diffable" means two runs of the system can be compared chunk-by-chunk, the way `diff` compares two versions of a text file line-by-line. If the corpus and config are unchanged, chunk ids from a run today match chunk ids from a run last month exactly, so a change in behavior can be attributed to a specific chunk rather than lost in the noise of everything having new, unrelated ids.

The human-readable prefix (`TSLA:1A:27:…` — ticker, item number, position, then a short hash) is a secondary convenience: it lets you recognize at a glance which filing and section a chunk id refers to when reading a mistake table, without looking anything up.

3.3 Index and retrieval: two retrievers, one catalog, one fingerprint

Whichever vector store is used, the system also keeps a plain local copy of every chunk's text (a file called `chunks.jsonl`) next to it, because keyword matching and the provenance graph both need the actual text of a chunk, not only its vector. ("Sidecar," used throughout this section, is a small companion file that rides alongside a main data store and records metadata *about* it — here, facts about how the index was built, kept separately so the main store doesn't need to be re-read or altered just to check them.) Three design choices in this layer are worth explaining:

- **Embedding fingerprint guard.** A sidecar file records which embedding model, model revision, and vector dimension were used to build the index. If someone later loads that index while a *different* embedding model is configured, the system stops with an explicit "Embedding mismatch" error instead of quietly returning wrong results. The reason this guard is needed comes from how vector search works: retrieval finds the chunks whose vectors are most similar to the query's vector, using **cosine similarity** — the cosine of the angle between two vectors, which measures whether they point in the same *direction* regardless of their length. This is the standard choice over Euclidean distance (straight-line distance between the vector endpoints) because these embedding vectors encode meaning primarily in their *direction*; two chunks about the same topic point the same way even if one is a slightly "longer" (larger-magnitude) vector than the other, and cosine similarity ignores that magnitude difference while Euclidean distance would be thrown off by it. (Nomic's embeddings are also L2-normalized, which makes cosine similarity and Euclidean distance equivalent in practice — but cosine is the conceptually correct measure regardless.) The guard matters because a similarity computed between vectors from two *different* models is still a valid-looking number between -1 and 1, yet meaningless, since the two models' vector spaces share no common notion of direction; only the recorded fingerprint reveals the mismatch.
- **Rank fusion, not score fusion.** BM25 scores and cosine similarities live on incomparable scales (BM25 scores are unbounded and corpus-dependent; cosine similarities are bounded between -1 and 1) — so combining them with a weighted sum of the raw scores would silently bake in an arbitrary calibration between two numbers that were never meant to be added together. Reciprocal-rank fusion [6] sidesteps this by combining each chunk's *rank* (1st, 2nd, 3rd, ...) in each retriever's result list, not the raw score — specifically `Σ w/(k + rank)` summed across retrievers, where a fixed constant `k` softens the difference between adjacent ranks. Ranks are always small integers on the same scale regardless of what produced them, so no calibration step is needed. The 0.6/0.4 lexical lean (favoring BM25) is domain signal, not an arbitrary choice — financial queries are dense with tickers, acronyms, and defined terms that exact keyword matching handles better than semantic embedding matching.
- **Precomputed BM25 sidecar.** BM25 over 280k chunks costs ~36 s of tokenization (breaking every chunk's text into indexed word-pieces) the first time a question is asked in a session; a one-time "finalize" step instead builds this sidecar once, in advance, directly from the Chroma store — the restart-proof source of truth — so a session's first question costs ~6 s instead. Deriving both the text catalog and the BM25 sidecar from the vector store itself (rather than re-reading the original input

files) means a crashed-and-resumed indexing run still produces consistent artifacts, because the store is the one place that reflects exactly what was actually indexed.

One profiling result reshaped the indexing tool: on this hardware the slow step by far is turning chunks into vectors (~10 chunks per second), while cutting the text into chunks (~20,000 per second) and writing them to the store (~900 per second) are effectively free. Counter-intuitively, running the embedding across multiple CPU processes makes it *slower* (8 workers manage only ~3 per second), because the work is limited by how fast data moves through memory rather than by raw compute, so extra processes merely compete for the same memory. The indexer therefore runs a single stream on the GPU, writes results on a background thread while the next batch embeds, and periodically clears the GPU's memory cache. The lesson: measure before parallelizing — here the obvious fix made things worse.

3.4 Generation and verification: deterministic invariants around a stochastic judge

The generator receives numbered evidence blocks and must cite `[n]`; each `[n]` the generator writes is matched back to the specific retrieved chunk it refers to, so a citation always points at a real chunk and is never just text the model made up. Verification then runs claim-level judging (`generation/verifier.py`), and the code's most important block is not the LLM call but what happens after it:

```
cv.valid_quote = self._quote_in_source(cv, context)
cv.numeric_ok = self._numeric_in_source(cv, context)
# Support must name its source: SUPPORTED/PARTIAL requires a citation
# whose quote/figures actually check out, else it is unverifiable.
if cv.verdict in (SUPPORTED, PARTIAL) and (
    cv.citation is None or not cv.valid_quote or not cv.numeric_ok):
    cv.verdict = UNSUPPORTED
# A contradiction must point at the refuting evidence; "contradicted
# by nothing" is judge shorthand for absent-from-evidence.
if cv.verdict == CONTRADICTED and cv.citation is None:
    cv.verdict = UNSUPPORTED
```

The rationale is an asymmetry in what each kind of verdict *does* downstream, not an asymmetry in how likely the judge is to be right. A judge's **negative** verdict (UNSUPPORTED) only ever triggers a defensive action: the claim gets flagged, the repair pass may drop it, or the answer may abstain. If the judge is wrong here — if it flagged a claim that was actually fine — the cost is a claim removed from an answer that could have kept it, or an unnecessary abstention. That is a real cost, but it is a cost paid *once*, borne entirely within this system, and it fails toward caution rather than toward presenting something false. A judge's **positive** verdict (SUPPORTED), by contrast, is what *licenses the system to show the user a citation and say "this is grounded."* If the judge is wrong here — if it says SUPPORTED but the claim isn't actually backed by that text — the error propagates all the way to the end user as a false assurance, which is the exact failure this whole verification layer exists to prevent. So a false negative degrades gracefully (an answer is more cautious than it needed to be); a false positive fails catastrophically (a false claim is delivered with a stamp of verification on it). That is why only positive verdicts get a mandatory, independent, mechanical check on top of the judge's say-so: the

supporting quote must be a **verbatim substring** of the cited chunk, and any figure asserted in the claim must appear in that chunk. These two tests — the quote-match and the number-match — are simple text comparisons, run without calling any model, and they exist specifically on the positive verdicts because a false positive is the expensive direction of error: they shrink the room in which the judge's biases (§10) could let an ungrounded claim through unnoticed. The same principle normalizes self-inconsistent verdicts: SUPPORTED-by-nothing and CONTRADICTED-by-nothing are both re-labeled UNSUPPORTED (the safe, defensive direction), because each asserts a relation to evidence while naming none, and an unverifiable assertion is treated as no assertion at all.

The judge is required to answer in a fixed structured format rather than free prose: each claim must have non-empty text, its verdict must be one of the four allowed labels, and it must include a citation field (a chunk number, or an explicit "none"). The prompt carries the one rule the schema cannot express — absence of evidence is UNSUPPORTED, never CONTRADICTED — and the invariant block above normalizes any verdict that is well-formed but internally inconsistent, such as a contradiction naming no refuting passage. For a small local model, a schema written like a contract (with `required` and `enum`) enforces far more reliably than prose instructions.

Two inference budgets are set by arithmetic, not left at their serving defaults, because an undersized budget truncates text silently instead of erroring. The **context window** must hold everything a model reads: for the generator, the 8 chunks × 3,600 characters (≈7k tokens at ~4 characters per token) plus the prompt; for the judge, the full answer plus all evidence (~5.4k tokens). Each of these two amounts is larger than Ollama's default 4,096-token limit, so the system raises the limit to 8,192 tokens (`num_ctx: 8192`) and recomputes it whenever the number of chunks or the chunk size changes. The **output budget** must hold the longest legitimate verdict: since every supported claim carries a verbatim quote, a dozen quoted claims run to roughly two thousand tokens, so the judge's `num_predict` is 2,048. One consequence of the output budget is worth stating because it is safety-relevant rather than obvious: a verdict truncated mid-JSON parses as *zero claims*, which would look like "nothing to verify," so the pipeline surfaces the zero-claims case explicitly in every artifact — an answer can never pass as verified simply because its verification came back empty.

The **gate and repair** logic completes the loop: if the supported fraction falls below the groundedness floor (0.6) or any claim is CONTRADICTED, the generator gets exactly one re-prompt listing the flagged claims and instructing their removal; the repaired answer is re-verified; persistent failure becomes an abstention whose *reason* is recorded. One repair round, not a loop — iterated self-correction tends to converge on confident paraphrase rather than grounding, and every additional round doubles inference cost.

3.5 The provenance layer: a graph that refuses to guess

The provenance-graph code (`provenance/dag.py`, ~250 lines) is deliberately simple: two small record types, one for a node and one for an edge, a routine that assembles them into a graph, and a set of exporters that render that graph as JSON, as a static diagram (Graphviz), and as an interactive web page. What matters is three things the assembler refuses to do:

```
ver = answer.get("extra", {}).get("verification") or {}
if not ver and not answer.get("abstained"):
    raise ValueError("answer has no verification report — run with "
                     "verification.enabled: true (the DAG is not built from guesses)")
```

First, **no report, no graph** — except for pre-generation abstentions, whose honest DAG is a question, the insufficient evidence that was retrieved, and a `rejected_conclusion` carrying the reason. Second, **no fabricated edges**: an UNSUPPORTED claim gets *no* evidence edge; it surfaces in the mistake table as `missing_evidence`. The temptation to draw a dotted "probably from here" edge is precisely the temptation to rationalize post-hoc, and the graph's audit value depends on resisting it — *absence is information*. Third, **uncited evidence stays in the graph**, rendered dashed and muted: passages the retriever fetched but no claim used are a retrieval-precision signal and occasionally the most interesting object on a slide (they show what the system *looked at and ignored*).

One serialization rule keeps the whole chain interpretable: **the verification report is self-contained**. Claim verdicts refer to evidence by position (`citation: 3` means "the third evidence block the judge saw"), so the report carries that evidence list itself — chunk ids, filing provenance, and quotes — inside the persisted answer (`answer.extra["verification"]["evidence"]`) rather than assuming the retrieval context still exists wherever the report is later read. Carrying the evidence inside the saved answer is what lets the graph, the deep-dive exhibits, and the mistake tables all be rebuilt later from that one file alone, long after the program that produced the answer has stopped running. The general principle: once a result is written to a file it must stand on its own, without needing the program that produced it — a report whose references point at things that no longer exist is not a usable report.

The exporters share one palette and one visual language (evidence → claims → conclusion, left to right; verdict badges; dashed = uncited) so that the interactive HTML used for exploration and the PDF/PNG used in slides are recognizably the same object. The interactive page carries its own data and loads the graph-drawing library from the web, so the project needs no extra Python drawing package. This reflects a deliberate rule: the core keeps almost no dependencies, and the automated tests run with none of the machine-learning or vector-database packages installed at all — which is what keeps the offline test path fast and honest.

3.6 Batch tooling: overnight runs assume failure

The gold-set runner (`tools/provenance_dag.py`) processes 260 questions at roughly 2–5 minutes each on a 16 GB M2 — an overnight commitment, engineered accordingly: per-question `try/except` that records an error row and continues (one Ollama timeout must not cost a night); `summary.csv` appended and flushed after every question (a crash at question 200 loses nothing); resume by artifact presence (a finished question's directory is skipped on restart); and the whole thing wrapped in `caffeinate` plus an auto-restart loop at launch. None of this is sophisticated; it is the standard precaution for any job that runs longer than someone will sit and watch it. The working assumption is that a 16-hour run *will* be interrupted at some point, so an interruption should cost nothing. Separately, the reporting and figure tools use no model at all — they read the files the run already wrote — so analysis can proceed while the long run is still using the GPU.

3.7 Metrics: proxies labeled as proxies

The report layer (`tools/provenance_report.py`) computes the corpus metrics and the per-question error attribution. Its `classify()` is an ordered decision list — placebo status first, then abstention, then retrieval-vs-gold, then wrong-period, then claim verdicts — encoding the diagnostic priority: *a generation symptom downstream of a retrieval failure is attributed to retrieval*. Two metrics are explicit proxies. Qualifier preservation works from a fixed list of hedging words — modal verbs such as *may* and *could*, and forward-looking verbs such as *expects* — matched only as whole words. Words with a common non-hedging meaning are deliberately left off the list (*estimates* appears all over ordinary accounting text; *certain* would match inside *uncertain*), a decision made after looking at which words were producing false matches. Every flagged claim, paired with the quote it hardened, is written to a CSV file so the number can be checked case by case. Wrong-period checking matches any 19xx/20xx year named in the question against cited filing dates with ±1 tolerance (a 10-K for fiscal *t* files in early *t+1*); the pattern covers both 19xx and 20xx because questions about pre-coverage decades are precisely how period-blindness presents (§5.4 shows one caught in the wild). Both proxies carry their limitations in their names and comments; the honest successor for each (a judged strengthening check; XBRL-anchored period grounding) is scoped as future work rather than half-implemented. The rule: **a crude measure whose crudeness is visible beats a sophisticated one whose failure modes are not**.

3.8 Tests: pin the failure modes that matter

The suite (56 tests, deterministic, offline — mock LLM, hashing embeddings, lexical judge) runs in ~2 s with no models. Its most valuable tests pin the failure modes this section identifies as consequential: word-boundary tests on the qualifier heuristic ("difficult" must not match "if"; a claim that keeps *any* hedge is not a hardening), a DAG test asserting the no-report refusal, and a round-trip test on serialization. Testing philosophy follows the architecture: everything stochastic is behind an interface with a deterministic stand-in, so CI exercises the *plumbing* completely and the *models* not at all — model quality is what the evaluation harness measures, not what unit tests assert.

4. Data

Corpus. S&P 500 filings from official SEC EDGAR (fair-access rate-limited, descriptive User-Agent), forms 10-K, 10-Q, and 8-K, fiscal years 2023–2025: approximately 4,200 filings roughly balanced across the three forms (~1,400 each). Text is extracted per filing with a JSON sidecar carrying issuer metadata (company, CIK, accession, filing date, fiscal year, section map). One known metadata gap is inherited by the whole pipeline: the production index was built from filename-derived metadata, so the `company` field is unpopulated (ticker, CIK, accession, form, and date are present) — citations therefore display tickers.

Index. 281,777 chunks at 3,600 characters (≈900 tokens) each, embedded with Nomic embed-text v1.5 (768-d, revision-pinned) into Chroma/HNSW, with the BM25 sidecar and chunk catalog derived from the store (§3.3). By form: ~194k chunks from 10-Ks, ~83k from 10-Qs, ~4.9k from 8-Ks — 10-Ks dominate because they are an order of magnitude longer.

Gold set. A 260-question subset of FinDER [39] (`eval/finder_semantic.json`), each with a target ticker, a category label (Legal, Governance, Risk, Company overview — 65 per category exactly), and one or more *verbatim relevant passages* from the filings serving as retrieval ground truth. All 260 are answerable by construction. Questions are terse and practitioner-flavored ("Legal proceedings & material liabilities loss range AXP") — FinDER was built specifically from short, acronym-heavy queries submitted by real financial professionals, which stresses lexical retrieval and makes them harder than conversational phrasings.

Negative controls. Six authored placebo questions (`eval/placebo.json`), one per unknowability type: a future filing (NVDA fiscal-2028 forecast), a pre-coverage period (Apple's 1999 risk factors), an unindexed private company (SpaceX), a false premise (Tesla's "2024 bankruptcy"), an uncovered form type (Microsoft's DEF 14A), and undisclosed granularity (AWS revenue on a specific day). Each is marked `answerable: false` and documents *why* it is unanswerable; correct behavior is refusal (§14).

Hardware and cost. All experiments ran on a single Apple M2 with 16 GB of memory. The generator and the judge take turns on the GPU because only one fits in memory at a time; a full pass over the 260 questions takes about 16–20 hours of real time, roughly 2–5 minutes per question including verification.

5. Results

Numbers below are from the **complete gold-set run (all 260 questions)** plus the 6 placebos. All claim-level figures are **judge-relative** (Mistral 7B under the §3.4 invariants) and, by the verbatim-quote design, lower bounds on true support.

5.1 Aggregate metrics

Of 260 questions: **150 answered, 110 abstained** (42.3% abstention). Across answered questions the judge decomposed **1,043 claims**: 770 supported, 273 unsupported, and — notably — **0 partial and 0 contradicted** (§6.3 discusses this collapse to a binary verdict distribution).

- **Faithfulness (claim support rate): 0.738** — of claims in delivered answers, ~three quarters carry a verified verbatim quote from a cited chunk.
- **Context recall vs gold passages: 0.262** — the retriever's top-8 covered the gold passage for barely a quarter of questions. This is the headline number: the bottleneck is retrieval, not generation.
- **Weighted Claim Error Rate: 0.523** (default weights: unsupported 2, partial 1, contradicted 3; supported 0).
- **Qualifier preservation: 0.551** — of 207 supported claims whose verified evidence quote is hedged, 114 kept a hedge and 93 stated the hedged content as bare fact. Nearly half of all hedged filing language hardens on its way into answers; every case is listed in `qualifier_drops.csv`.
- **Wrong-period answers:** 1 of 2 answered year-named questions cited filings outside ±1 year of the asked year (the gold set names years rarely; the placebo set probes this deliberately).

5.2 Error attribution

Per-question dominant error source over the 260:

- retrieval miss → abstained: **100** (38%)
- retrieval miss → ungrounded claims: **63** (24%)
- generation: unsupported claims: **46** (18%)
- clean: **27** (10%)
- generation: dropped qualifiers: **14** (5%)
- abstained (gate/judge): **10** (4%)

Read as subsystems: **~63% of questions fail at retrieval**, ~23% in generation, ~4% at the verification gate, and 10% pass end-to-end. The two retrieval-failure modes differ in cost: the abstaining kind is safe (the system says "I don't know"), while the ungrounded-claims kind produces confident answers built on tangential passages — the most dangerous cell in the table, and the one only the paired metrics (support × gold-recall) expose. Clean-rate is uniform across question categories (Legal 7/65, Governance 5/65, Risk 8/65, Company overview 7/65), suggesting the retrieval bottleneck is not category-specific but a general vocabulary/phrasing mismatch between terse analyst queries and filing language.

5.3 Case studies (the five seminar exemplars)

Each has a slide-ready figure (`out/provenance/<id>/dag.pdf`) and a full write-up in `showcase.md`.

- **finder_0020 (IR) — clean.** 8/8 claims supported, each with a verbatim quote; two retrieved-but-uncited chunks visible as dashed nodes. The baseline for what "verified" looks like.
- **finder_0017 (ADBE) — generation flagged.** Gold recall 1.0, yet 2 of 7 claims unsupported; the DAG isolates exactly which sentences to distrust while the rest stand.
- **finder_0002 (FANG) — grounded-looking drift.** Gold recall 0.0; the generator built a fluent answer on tangential passages (guarantee/accounting boilerplate instead of the asked-about legal proceedings). Claims *look* cited; the conjunction of recall-0 and the DAG shows the answer is off-target.
- **placebo_0002 (AAPL 1999) — period-blind confabulation.** See §5.4.
- **finder_0001 (ES) — honest refusal.** Gold evidence never retrieved; the model itself declared insufficient evidence. The DAG: one dashed evidence node, a red rejected-conclusion card, zero claims.

5.4 Negative controls

5 of 6 placebos correctly refused (future filing, unindexed company, false premise, uncovered form, undisclosed granularity). The sixth is the most instructive artifact of the study: asked for **Apple's risk**

factors from its 1999 10-K — 24 years before the corpus begins — the system answered at confidence 0.85 by quoting Apple's **2023 10-Q** boilerplate, with **6/6 claims judged SUPPORTED and every verbatim quote verified**. Every layer of the faithfulness stack passed — retrieval, citation, claim support, quote verification — and the answer is wrong, because no layer checks the *period*. Faithfulness ≠ correctness, demonstrated end-to-end in one figure (`out/placebo/placebo_0002/dag.pdf`: all-green claims hanging off a 2023 evidence node under a 1999 question).

The same control also made a methodological point: it initially slipped past the wrong-period metric too, whose year pattern matched only 20xx — a reminder that **negative controls audit the evaluator, not just the system**.

6. Discussion

6.1 Retrieval is the bottleneck, and the gold set says why. Context recall of 0.262 against verbatim gold passages, with 63% of per-question failures attributed to retrieval, locates the dominant loss well before generation. Two forces compound: terse, acronym-dense analyst phrasing (favoring BM25, but only when the query's terms actually appear in the passage) and single-locus gold (§12's caveat — recall-vs-gold under-credits legitimately grounded answers from non-gold passages). Both point the same direction for remediation: query expansion/rewriting before fusion, and multi-locus gold before concluding the retriever is as weak as 0.262 suggests.

6.2 Faithfulness machinery cannot see time. The placebo confabulation (§5.4) is the study's sharpest single result: a complete pass of retrieval-grounding-citation-verification checks on a wrong answer. Support relations are timeless — a 2023 passage genuinely supports "the Company's business can be affected by many factors" — so period correctness must be its own dimension, checked against the *question's* temporal scope, not the evidence's internal consistency. The rigorous path is to anchor on XBRL (eXtensible Business Reporting Language — the machine-readable standard in which filers tag each financial figure together with an explicit context, including the reporting period it belongs to), so an asserted number can be checked against the period the question actually asked about (§15); the interim proxy at least makes the failure countable. The general form of the lesson: every verification stack has a blind axis, and negative controls are how you find yours.

6.3 The judge is strict, binary, and uncalibrated — and the metrics inherit it. The verdict distribution collapsed to SUPPORTED/UNSUPPORTED: PARTIAL went unused by the judge, and CONTRADICTED was eliminated entirely by the citation-required invariant (every contradiction the judge asserted named no refuting passage and was normalized to UNSUPPORTED). Manual inspection of unsupported verdicts (e.g., the NVDA gate case, where 5 "unsupported" claims sit beside on-point quotes) suggests systematic under-crediting. Three consequences: faithfulness (0.738) is a floor; the 42.3% abstention rate is partly gate-induced rather than evidence-induced; and Weighted Claim Error Rate currently runs on two effective weight levels. None of this invalidates the *diagnostic* use of the verdicts (the error attribution is robust to a uniformly strict judge), but the absolute numbers should not be quoted without the judge-relative qualifier until the human-calibration protocol

(`docs/HUMAN_EVAL.md`) has been run.

6.4 Abstention is doing real safety work. 5/6 placebo refusals plus 100 retrieval-miss abstentions on the gold set show the gates fail *closed*: when the system lacks evidence it predominantly says so. The cost is coverage (42.3% abstention on answerable questions is high), and the risk–coverage curve (§14) is tunable via the evidence and groundedness floors. The one placebo that slipped through did so precisely where the gates have no sensor (time), reinforcing §6.2.

6.5 Threats to validity. Single generator and single judge, both small local models — findings about *their* error rates need not transfer, though the framework is model-agnostic by construction. Gold set of one style (terse FinDER queries) and single-locus passages; clean-rate uniformity across categories suggests but does not prove phrasing-driven retrieval failure. Qualifier preservation is a lexical proxy (upper bound, §15); faithfulness is judge-relative (lower bound, §9); the two biases point in opposite directions and are stated with every number. The placebo set is small (6) and authored by the system's builders — an adversarially constructed placebo battery is future work.

6.6 What follows. In priority order: (1) judge calibration against a hand-labeled claim sample, then judge ensembling (LLM + NLI, §8) with disagreement surfaced rather than averaged; (2) retrieval work — query rewriting, and re-scoring recall against multi-locus gold; (3) XBRL-anchored period and figure checking, populating the reserved `calculates` edges; (4) gold error weights for WCER from analyst annotation (which mistakes actually matter); (5) the `company` metadata backfill from sidecars, a one-pass cosmetic fix.

7. Claim decomposition: the atomic unit of trust

The sections that follow step back from this particular system and place each of its design choices in the research literature it draws on; a reader who wants the ideas without the code can start here. We begin with the idea underlying the whole verification layer: evaluating an answer one claim at a time.

The move from answer-level to claim-level evaluation is the field's most consequential methodological shift. FActScore (Min et al., 2023) [8] made it canonical: decompose generated text into *atomic facts*, verify each against a knowledge source, and report the supported fraction. Why atomicity matters: verification is a *binary relation* between a statement and evidence, and it is only well-defined when the statement asserts one thing — a compound sentence can be half-true, and any verdict on it is an average in disguise. Decomposition therefore does for evaluation what normalization does for databases: it makes the unit of judgment match the unit of fact. FActScore also exposed the recipe's costs: granularity is itself a modeling choice (split too fine and claims lose the context that fixes their meaning; too coarse and verdicts blur), and error compounds when decomposer and verifier are both LLMs.

RAGAS (Es et al., 2023) [9] packages this into the standard quartet: **faithfulness** (fraction of answer claims supported by retrieved context), **answer relevancy**, **context precision**, and **context recall**. Two critiques matter beyond the paper. First, RAGAS metrics are LLM-computed with prompt-sensitive definitions, so scores are not comparable across judge models or versions; treating a RAGAS number

as a property of the *system under test* alone is a category error — it is a property of the (system, judge) pair. This project pins a local judge and layers deterministic checks so that the judge-dependent surface is as small as possible, and reports metrics explicitly as judge-relative (§5). Second, faithfulness is *support against retrieved context*, not truth: a faithful answer over badly retrieved context is faithfully wrong — the §5.3 drift case and the §5.4 placebo are both instances, and the dashboard pairs claim support with gold-passage recall because the combination distinguishes what no single metric can.

RAGChecker (Ru et al., 2024) [10] pushes decomposition into *diagnosis*: by checking answer claims against both retrieved context and a gold answer, it separates retriever errors from generator errors and quantifies noise sensitivity. Its taxonomy is the nearest published relative of this project's per-question error attribution (§5.2); the project's version trades generality for domain specificity and for attribution simple enough to put on a slide. ARES (Saad-Falcon et al., 2023) [11] addresses judging cost — train lightweight judges on synthetic data, then correct their aggregate scores with prediction-powered inference (a statistical technique that combines a few human labels with many model labels to recover valid confidence intervals) — and is the right reference when asked how claim-level evaluation scales beyond a local 7B judge.

8. Faithfulness before LLM judges: the NLI lineage

Claim verification predates instruction-tuned judges, and the older tools remain relevant both as baselines and as design vocabulary. The workhorse of that older generation is **natural language inference (NLI)** — a model trained to decide, for a pair of sentences, whether the first *entails* the second (if the first is true the second must be too), *contradicts* it, or is *neutral*. Verification maps directly onto this: put the evidence as the first sentence and a claim as the second, and "is the claim supported?" becomes "does the evidence entail the claim?". The tools below are variations on that move. FactCC (Kryściński et al., 2020) [12] trained a consistency classifier on synthetically corrupted summaries — the insight that *you can manufacture supervision for inconsistency* by perturbing faithful text. SummaC (Laban et al., 2022) [13] showed that off-the-shelf sentence-pair NLI, aggregated sentence-by-sentence rather than document-level, is competitive — the aggregation granularity, not the model, was the missing piece (the same granularity argument as §7, arrived at independently). QAFactEval (Fabbri et al., 2022) [14] verified by round-trip: generate questions from the summary, answer them from the source, compare answers — indirection that catches paraphrase-level support that string matching misses.

Why this lineage matters here: NLI-style entailment is the natural next partner for the LLM judge (§6.6) — cheaper, deterministic, bias-free with respect to instruction-following pathologies — at the known cost of weakness on multi-sentence aggregation and numbers (which this system covers with its own numeric check). SelfCheckGPT (Manakul et al., 2023) [15] is the reference-free alternative (sample the generator repeatedly; inconsistent statements are suspect) — relevant when there is no evidence corpus, dominated in RAG settings where there is. Chain-of-Verification (Dhuliawala et al., 2023) [16] moves checking into generation itself; the project's single repair pass is a one-round, externally-judged variant, on the argument (§3.4) that iterated self-correction converges on fluency

rather than grounding.

9. Attribution: citations that actually support

Verification (§7–§8) asks whether a claim is backed by the evidence *at all*; attribution asks the sharper question of whether the *particular source the answer points to* actually backs it. A citation is a claim about a claim — "this passage supports that statement" — and it can be false. Rashkin et al.'s *Attributable to Identified Sources* framework [17] made the standard precise with an operational test: a statement is attributable iff a generic reader, shown the cited source, would agree "according to the source, S". Bohnet et al. [18] built and measured attributed-QA systems at scale and found correctness and attribution dissociate — systems are frequently right for the wrong (cited) reasons, which is disqualifying in compliance settings even when the answer happens to be true. ALCE (Gao et al., 2023) [19] gave the field its automatic proxies: **citation recall** (every statement covered by its citations) and **citation precision** (each citation genuinely supports its statement), computed with NLI.

The project's strictness — SUPPORTED requires a *verbatim substring* quote, not an entailment call — is a deliberate position in this space. NLI-based citation precision accepts paraphrase support but inherits NLI's error modes on long premises and numbers, and its verdicts are not human-checkable at a glance. Substring support is narrower (real paraphrase support gets downgraded — measured faithfulness becomes a *lower bound*, as §5's numbers are labeled) but every positive verdict ships its own proof: the quote either is in the chunk or is not. In a domain whose operative standard is "show me the sentence in the filing", auditability was priced above coverage. The asymmetry is intentional and should be stated with the numbers: this system under-credits support by design and never over-credits it by construction.

10. LLM-as-judge: power and pathologies

The judge at the center of this system is itself a language model, which invites the obvious objection: can one language model be trusted to grade another? The honest answer is *only under constraints*, and the constraints follow from what is known about the practice. Using a strong LLM to grade outputs (Zheng et al., 2023 [20]) is now default practice, and its pathologies are documented: position bias, verbosity bias, poor discrimination near ties, and **self-preference** — models favor their own generations and can recognize them (Panickssery et al., 2024 [21]). G-Eval (Liu et al., 2023) [22] showed prompted judges track humans on summarization while inheriting their prompts' biases.

The project's three judge-design rules map one-to-one onto these findings. **Independence** (judge from a different model family, enforced by a startup guard that refuses generator==judge): self-preference survives paraphrase, so "different checkpoint" is not enough — and beyond bias, *correlated failure* is the deeper argument: a generator and judge sharing training lineage share blind spots, and verification's value is bounded by the independence of its errors. **Determinism where possible** (§3.4's invariants): every mechanically checkable component of a verdict is checked mechanically, so the judge's discretion — and therefore its biases — apply only to the residue that genuinely requires judgment. **Schema constraint with required fields**: the judge speaks a typed protocol, not prose, because parsing

failures are silent verification failures (§3.4).

What remains open — here and field-wide — is **calibration against humans**: the repository's `docs/HUMAN_EVAL.md` protocol (label a claim sample, measure judge agreement, compare lexical vs LLM judge) is the gate before claim-support numbers are quoted as ground truth; §6.3's evidence of systematic under-crediting makes this the top-priority follow-up. Until then, every claim-level number carries the qualifier *judge-relative* — practicing on the metrics the same hedge-preservation the system demands of its answers.

11. A taxonomy of hallucination, and which kinds this system catches

Every verdict the judge assigns is, in the end, a hallucination label — so it helps to have a map of the kinds of hallucination there are, and to be explicit about which kinds this system can and cannot catch. The surveys (Ji et al., 2023 [23]; Huang et al., 2023 [24]) distinguish **intrinsic** hallucination (contradicts the source) from **extrinsic** (unverifiable against the source), and orthogonally **faithfulness** (consistency with provided context) from **factuality** (consistency with the world). RAG evaluation is mostly a faithfulness affair — retrieved context is the contract — but filings add a factuality edge: a claim can faithfully repeat a forward-looking statement and be false about the world, which is why preserving the filing's hedges is a correctness property here, not style (§15).

Mapped to this system: intrinsic → CONTRADICTED (with the refuting passage required by invariant); extrinsic → UNSUPPORTED plus the numeric check (numeric hallucination is the highest-severity extrinsic class in finance and the one surface-overlap judges miss most). The DAG additionally makes visible two classes the binary taxonomy has no name for. **Topic drift under retrieval failure** — every claim individually supported by *some* retrieved passage, the ensemble answering a different question than asked; its signature is a conjunction (gold recall ≈ 0, claim support high), invisible to any single metric (§5.3, case 3). And **period-blind support** — claims supported by evidence from the wrong reporting period (§5.4), where the support relation itself is sound and the temporal binding is the error.

12. Retrieval evaluation and hybrid search

The retrieval half is classical information retrieval (IR), and its standard measures apply, with one domain twist. Those measures ask, in different ways, whether the right passages surface near the top of the ranked results: **Recall@k** (did a relevant passage appear anywhere in the top *k*?), **MRR** (mean reciprocal rank — how near the top the first relevant result sits, averaged over questions), and **MAP@k** (mean average precision — rewarding many relevant results ranked high, not just the first) [26]. The twist: FinDER-style gold relevance is expressed as *verbatim passages*, not document ids, so the harness scores retrieval by token-overlap between retrieved chunks and gold passages (threshold 0.5). Why overlap rather than ids: chunking is a system parameter, and id-based relevance judgments (qrels, the IR term for a labeled list of which documents count as relevant to which query) break every time chunk boundaries move; passage overlap is invariant to re-chunking, at the cost of coarseness. The caveat that §5's numbers inherit: **recall-versus-gold is a retrieval diagnostic, not a correctness verdict** —

FinDER's gold marks *one sufficient* evidence locus, not all loci, so an answer can be legitimately grounded in text the gold never mentions, and 0.262 is best read as a lower bound on useful retrieval.

Hybrid design rationale: BM25 [3] wins on the exact-match vocabulary of finance (tickers, acronyms, defined terms); dense retrieval [25] wins on paraphrase ("liquidity concerns" vs "ability to meet obligations"); RRF [6] combines them at rank level to avoid cross-scale score calibration (§3.3). Top-k is 8 with large (3,600-char) chunks — a "fewer, better, bigger" choice supported by Liu et al.'s positional-degradation results [7]: models attend poorly to the middle of stuffed windows, so past a modest k, added context is noise the judge must later discount. Hard truncation (§3.4's context-budget arithmetic) is the degenerate endpoint of the same phenomenon, which is why the effective context budget is treated as a first-class, derived configuration value rather than a serving default.

13. Provenance graphs: the intellectual lineage of the DAG

Three traditions converge in the claim-provenance DAG, and knowing them answers "why a graph at all?"

Argumentation theory. Toulmin (1958) [27] decomposed practical argument into claim, data, warrant, backing, **qualifier**, and rebuttal — against the then-dominant view of argument as logical deduction, arguing that real justification is *field-dependent* and defeasible. The DAG is a computational Toulmin diagram: evidence nodes are data; a `supports` edge is a warrant whose backing is the verbatim quote (the warrant made checkable); `rejected_conclusion` encodes rebuttal; and qualifier preservation (§15) operationalizes Toulmin's point that an argument minus its qualifier is a *different and stronger* argument than the evidence licenses. This is a fifty-year-old answer to why hedge-dropping is a correctness failure, not a style issue.

Data provenance. W3C PROV [28] standardizes provenance as a typed graph over entities, activities, and agents (`wasDerivedFrom`, `wasGeneratedBy`), built for the question "where did this artifact come from and should I trust it?" The DAG applies PROV's discipline to a reasoning pipeline: every node carries source identity (CIK, accession, form, date, item), every edge is typed, and the graph records *what happened*, not a plausible reconstruction. PROV is also the interoperability target if the system ever feeds institutional lineage or audit tooling.

Structured NL inference. EntailmentBank (Dalvi et al., 2021) [29] and successors build multi-step entailment trees from evidence to hypothesis. The DAG is deliberately *shallower* — one hop evidence→claim, one claim→conclusion — because every added inferential layer is another surface an LLM can fabricate: a beautiful multi-hop tree whose intermediate nodes were invented by the explainer is worse than a flat graph that is true. The schema reserves `intermediate_claim`, `calculates`, and `compares` for a future in which those hops are *produced by the pipeline* (e.g., the XBRL layer of §6.6) rather than reconstructed by an explainer. The governing principle, stated once more because it is the design's core: **absence is information** — a claim with no evidence edge is the graph telling the truth about the answer.

14. Abstention and selective prediction

A system that answers everything measures only accuracy; one that may refuse measures the **risk–coverage trade-off** (Kamath et al., 2020 [30]): at what answer rate is what error rate achieved, and does confidence order errors correctly? The project abstains at three distinct points, each with different information available: pre-generation (evidence score below floor — cheapest, catches retrieval failure before spending generation), post-verification (groundedness floor / contradiction — catches generation failure), and model self-report ("insufficient evidence" — catches what the gates' thresholds miss). Refusals are first-class outputs: each becomes a `rejected_conclusion` node with its reason, because *why the system refused* is exactly what an auditor asks and what a threshold-tuning engineer needs. §5's 110 abstentions and §6.4's reading of them are the empirical face of this design.

Evaluation must then include **negative controls** — the placebo set (§4): questions unanswerable from the corpus *by construction*, one per unknowability type. Each type isolates a distinct capability: the false-premise placebo tests presupposition rejection, the uncovered-form placebo tests whether the system knows its corpus boundary, the future-filing placebo tests temporal self-awareness. Correct behavior is refusal; a confident answer is confabulation whose DAG shows what tangential evidence it leaned on (§5.4). This is the unanswerable-question tradition of SQuAD 2.0 [31] transplanted to RAG, plus the calibration literature's insistence [32] that confidence be measured where the model *should* be uncertain. Without placebos, "correct abstention rate" is undefined and abstention thresholds are tuned blind.

15. Financial-domain specifics: hedges, safe harbors, and periods

Two project metrics have no generic-RAG counterpart; both are domain requirements with legal roots.

Qualifier preservation. Filings hedge deliberately and consequentially. The PSLRA safe harbor (1995) [33] shields forward-looking statements from liability *conditional on* meaningful cautionary language — the *may*, *could*, *expects*, *subject to* are load-bearing legal operators, and Item 1A risk factors are by construction statements about what *could* happen. A RAG answer that renders "competitive pressure could adversely affect us" as "competition exists" has strengthened the claim beyond what the filer asserted — a faithfulness failure invisible to entailment-style checking, since the weaker statement is entailed by the stronger reading. The metric: among supported claims whose verified quote contains a hedge, the fraction whose claim retains *any* hedge — 0.551 over the full run (114 of 207; §5.1), i.e., nearly half of hedged filing language hardens in generation. Implementation is a lexical proxy with its false positives engineered out by inspection (§3.7); the linguistics literature on hedging (Hyland [34]) and the Loughran–McDonald modal-strength word lists [35] are the upgrade path, and a judged "does the claim assert more strongly than the quote?" check is the principled successor. Note the metric's direction of error: lexical hedging detection under-flags (misses hedges phrased without lexical markers), so reported preservation is an *upper* bound — opposite in direction to the faithfulness lower bound of §9, and both directions are stated with the numbers.

Wrong filing / wrong period. Financial questions are period-anchored; filings are period-labeled; confusing them yields supported, fluent, wrong answers — §5.4 is the existence proof, at confidence 0.85 with every check green. The current proxy — years named in the question vs cited filing dates, ±1 year — is narrow and says so, reporting its denominator. The rigorous path is XBRL: asserted figures matched to tagged facts whose contexts carry explicit period semantics, connecting to the numerical-QA line of FinQA [36] and TAT-QA [37]. The reserved `calculates` edge type is the DAG's hook for that layer.

Benchmarks. FinanceBench (Islam et al., 2023) [38] demonstrated frontier models fail or refuse on a large fraction of straightforward filings questions — the empirical license for building verification-first. This project's gold set (§4) is a 260-question subset of FinDER (Linq AI Research, 2025) [39], purpose-built for RAG evaluation over 10-Ks with real, terse, acronym-heavy financial-professional queries; its single-locus gold (one marked passage per question, not all sufficient passages) is the source of the recall-≠-grounding caveat (§12).

16. How the pieces answer the seminar's question

The seminar claim: *aggregate metrics and provenance graphs are complements, not substitutes.* Claim-level metrics (FActScore, RAGAS, RAGChecker) made hallucination measurable but reduce to numbers whose causes are invisible; attribution work (AIS/ALCE) made citations checkable but stops at the claim–citation pair; RAGChecker diagnoses at corpus level. The DAG occupies the remaining cell: per-question, end-to-end, audit-grade explanation, where every aggregate number is one click from the evidence spans, verdicts, and gates that produced it — including for refusals and placebos, where the interesting object is what the system *didn't* do. The results make the complementarity concrete: the same faithfulness number (0.738) covers clean answers, flagged answers, and period-blind confabulation; only the paired diagnostics and the graphs distinguish them.

Limitations to state plainly, each with its direction of error: claim support is judge-relative and (by the verbatim-quote design) a lower bound; qualifier preservation is a lexical proxy and an upper bound; wrong-period checking covers only year-named questions; gold-recall conflates "missed the gold" with "grounded elsewhere"; verification costs one extra model call per answer; and the judge remains uncalibrated against humans pending the HUMAN_EVAL protocol. None are hidden — each is visible in the artifacts, which is the design's actual thesis.

17. Suggested reading order

- **RAGAS** [9] *(read)* — the shared metric vocabulary.
- **FActScore** [8] — claim decomposition done carefully; read for the granularity discussion.
- **RAGChecker** [10] — the published relative of the error-attribution table (§5.2).
- **ALCE** [19] + **AIS** [17] — what "supports" should mean; compare with §9's verbatim-quote position.

- **MT-Bench** [20] + **self-preference** [21] — why the judge is a different model family, by guard.
- **SummaC** [13] or **QAFactEval** [14] — the NLI alternative; candidate ensemble partner (§6.6).
- **Lost in the Middle** [7] — pairs with the context-budget arithmetic of §3.4 — positional degradation and hard truncation are the same failure at two severities.
- **Kamath et al.** [30] + **SQuAD 2.0** [31] — abstention as measured capability; the placebo design's ancestry.
- **FinanceBench** [38] and **FinDER** [39] — why filings QA needs all of the above, and where this project's own gold set comes from.
- **Toulmin** [27], ch. 3 — the fifty-year-old blueprint for the node and edge types.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. arXiv:2005.11401.

[2] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[3] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

[4] Nussbaum, Z., Morris, J. X., Duderstadt, B., & Mulyar, A. (2024). Nomic Embed: Training a Reproducible Long Context Text Embedder. arXiv:2402.01613.

[5] Malkov, Y. A., & Yashunin, D. A. (2016). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE TPAMI*, 42(4). arXiv:1603.09320.

[6] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR 2009*.

[7] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*. arXiv:2307.03172.

[8] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *EMNLP 2023*. arXiv:2305.14251.

[9] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *EACL 2024 demo*. arXiv:2309.15217.

[10] Ru, D., Qiu, L., Hu, X., Zhang, T., Shi, P., Chang, S., Jiayang, C., Wang, C., Sun, S., Li, H., et al. (2024). RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. *NeurIPS 2024 Datasets & Benchmarks*. arXiv:2408.08067.

[11] Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2023). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *NAACL 2024*. arXiv:2311.09476.

[12] Kryściński, W., McCann, B., Xiong, C., & Socher, R. (2020). Evaluating the Factual Consistency of Abstractive Text Summarization. *EMNLP 2020*. arXiv:1910.12840.

[13] Laban, P., Schnabel, T., Bennett, P. N., & Hearst, M. A. (2022). SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. *TACL 2022*. arXiv:2111.09525.

[14] Fabbri, A. R., Wu, C.-S., Liu, W., & Xiong, C. (2022). QAFactEval: Improved QA-Based Factual Consistency Evaluation for Summarization. *NAACL 2022*. arXiv:2112.08542.

[15] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *EMNLP 2023*. arXiv:2303.08896.

[16] Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., & Weston, J. (2023). Chain-of-Verification Reduces Hallucination in Large Language Models. *ACL Findings 2024*. arXiv:2309.11495.

[17] Rashkin, H., Nikolaev, D., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., & Reitter, D. (2021). Measuring Attribution in Natural Language Generation Models. *Computational Linguistics*, 49(4). arXiv:2112.12870.

[18] Bohnet, B., Tran, V. Q., Verga, P., Aharoni, R., Andor, D., Soares, L. B., Ciaramita, M., Eisenstein, J., Ganchev, K., Herzig, J., et al. (2022). Attributed Question Answering: Evaluation and Modeling for Attributed Large Language Models. arXiv:2212.08037.

[19] Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling Large Language Models to Generate Text with Citations (ALCE). *EMNLP 2023*. arXiv:2305.14627.

[20] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS 2023 Datasets & Benchmarks*. arXiv:2306.05685.

[21] Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations. *NeurIPS 2024*. arXiv:2404.13076.

[22] Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *EMNLP 2023*. arXiv:2303.16634.

[23] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12). arXiv:2202.03629.

[24] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM TOIS 2025*. arXiv:2311.05232.

[25] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*. arXiv:2004.04906.

[26] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

[27] Toulmin, S. E. (1958). *The Uses of Argument*. Cambridge University Press.

[28] Moreau, L., & Missier, P., eds. (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.

[29] Dalvi, B., Jansen, P., Tafjord, O., Xie, Z., Smith, H., Pipatanangkura, L., & Clark, P. (2021). Explaining Answers with Entailment Trees. *EMNLP 2021*. arXiv:2104.08661.

[30] Kamath, A., Jia, R., & Liang, P. (2020). Selective Question Answering under Domain Shift. *ACL 2020*. arXiv:2006.09462.

[31] Rajpurkar, P., Jia, R., & Liang, P. (2018). Know What You Don't Know: Unanswerable Questions for SQuAD. *ACL 2018*. arXiv:1806.03822.

[32] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *ICML 2017*. arXiv:1706.04599.

[33] Private Securities Litigation Reform Act of 1995, 15 U.S.C. §78u-5 (safe harbor for forward-looking statements).

[34] Hyland, K. (1998). *Hedging in Scientific Research Articles*. John Benjamins.

[35] Loughran, T., & McDonald, B. (2011). When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *Journal of Finance*, 66(1), 35–65.

[36] Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., Moussa, R., Beane, M., Huang, T.-H., Routledge, B., & Wang, W. Y. (2021). FinQA: A Dataset of Numerical Reasoning over Financial Data. *EMNLP 2021*. arXiv:2109.00122.

[37] Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., & Chua, T.-S. (2021). TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. *ACL 2021*. arXiv:2105.07624.

[38] Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., & Vidgen, B. (2023). FinanceBench: A New Benchmark for Financial Question Answering. arXiv:2311.11944.

[39] Linq AI Research. (2025). FinDER: Financial Dataset for Question Answering and Evaluating Retrieval-Augmented Generation. *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF 2025)*. arXiv:2504.15800.

Note on the FinDER gold set: `eval/finder_semantic.json` *(260 questions) is a subset of the full FinDER benchmark [39] — 5,703 expert-annotated query–evidence–answer triplets over real 10-K filings, published by Linq AI Research (Hugging Face:* `Linq-AI-Research/FinDER`*). The full benchmark's own stated design goal — short, acronym-heavy queries reflecting real financial-professional search behavior — is exactly what §4 and §9 describe as stressing lexical retrieval; that framing is the dataset's, not an inference.*